

Production ML Study Guide

Chapter 1: Pre-Processing

Pre-processing in machine learning refers to the series of steps applied to raw data to prepare it for modeling. This step is crucial because the quality of the data directly affects the performance of machine learning algorithms. Garbage in = Garbage out. [GIGO](#)



Key Pre-processing Steps:

1. **Handling Missing Data** (Imputation)
2. **Data Transformation** (Scaling)
3. **Encoding Categorical Variables:**
4. **Feature Extraction and Selection:**
 - Reducing the dimensionality of the dataset (e.g., through Principal Component Analysis, or selecting the most important features) can prevent overfitting and speed up the training process.
5. **Dealing with Outliers:**
 - Outliers can heavily influence some models (e.g., linear regression). Handling outliers (e.g., by capping/extending values) can improve model robustness.
6. **Balancing the Dataset:**
 - In classification tasks with imbalanced classes, the minority class can be underrepresented. Techniques like oversampling (e.g., [SMOTE](#)) or undersampling are used to address this issue.

Importance of Pre-processing:

- **Improves Model Accuracy:** Clean and well-prepared data helps models learn patterns more effectively.
- **Prevents Bias:** Pre-processing ensures that models don't learn from noisy or biased data, which could otherwise lead to incorrect predictions.
- **Enhances Model Computational Efficiency:** With reduced noise, fewer features, and better feature scaling, models train faster and perform better.

Chapter 2: Imputation

[Imputation](#) is the process of filling in missing data with substituted values. It's important because many machine learning algorithms cannot handle missing data and need complete datasets for training.

Types of Imputation:

- **Mean/Median Imputation:**
 - Replaces missing values with the mean (for normally distributed data) or median (for skewed data) of the column.
 - **Pros:** Simple, works well for continuous, *numeric* data.
 - **Cons:** Can distort the dataset if there are many missing values or if the data is not representative.
- **Most Frequent (Mode) Imputation:**
 - Replaces missing values with the most common value in the column.
 - **Pros:** Works well for *categorical* data.
 - **Cons:** Can introduce bias if the frequent value over-represents certain categories.

When Imputation Is Not Appropriate:

- **Too Many Missing Values:** If a column has a large percentage of missing data, imputing it could skew the dataset. In such cases, it's better to drop the column.
- **Irrelevant Feature:** If the column does not contribute meaningfully to the model, it's better to drop it instead of worrying about imputing it.

NOTE: Boolean Column

When dropping a column because of null values, it can be appropriate to make a boolean column to represent which rows had missing values. We did this with "has_cabin" in Titanic.

- **Usefulness:** A boolean column indicating whether a value was missing (like: `has_cabin`) can capture useful information for the model, as *missingness itself may be predictive*. It preserves the fact that a value was missing, even if the original column is later dropped.

Chapter 3: Encoding

Encoding is essential because most machine learning algorithms require numerical input, so categorical variables need to be transformed into numerical form.

Label (Ordinal) Encoding

Label Encoding assigns a unique integer to each category.

- **Pros:** Simple and works well when categorical values have an inherent order (ordinal data). Typically useful for binary categories that can be represented as 0/1.
- **Cons:** For nominal data (no order), the integer values can mislead the algorithm into thinking there's a meaningful ranking, causing poor performance.

One-Hot Encoding

[One-Hot Encoding](#) creates a binary (0 or 1) column for each category in the original column.

- **Pros:** Avoids introducing ordinal relationships between categories. Works well with nominal data. This is the PRIMARY encoding technique in most workflows.
- **Cons:** Increases dimensionality, especially with columns that have many unique categories, which can slow down training and lead to overfitting.
- I recommend doing OHE when non-binary (more than 2) categories are present

Binning for One-Hot Encoding

- When faced with many categories (12+), it might be useful to group categories together, or to create an "Other" category. Sample size matters.
- Binning simplifies the feature and can reduce noise

Dummy Variables

Dummy variables are the resulting binary columns from One-Hot Encoding, where each represents a category. You might also hear OHE as Dummy Encoding, or `pd.get_dummies()`

BONUS: Target Encoding

[Target Encoding](#) replaces each category with the mean of the target variable for that category. It's commonly used in classification and regression tasks.

- **Pros:** Can capture information about the relationship between a category and the target variable, often improving model performance.
- **Cons:** Prone to **overfitting** if the dataset is small or has many categories. Cross-validation or regularization is often required to mitigate this.

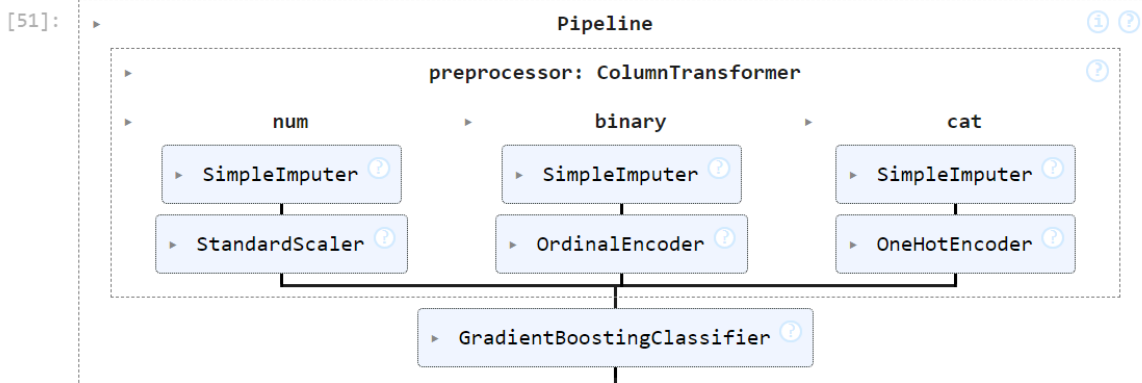
Workflow

For Numerical data -> First Impute (Mean/Median), then Scale

For Binary or Boolean data -> First Impute (Mode), then Label (Ordinal) Encode

For Categorical data -> First Impute (Mode), then One-Hot Encode

```
[51]: # Retrain the model on the ENTIRE dataset
gb = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', GradientBoostingClassifier(random_state=42))
])
gb.fit(X, y)
```



Chapter 4: Model Saving

Model saving involves storing a trained machine learning model to disk so it can be reused later without retraining. This is crucial for deploying models in production environments or for sharing with others. Enables deployment in real-world applications like your Capstone project.

Pickling a Model

[Pickling](#) is the process of serializing a model (converting it into a byte stream) so it can be saved to disk and loaded back later.

- **Libraries:**
 - **pickle**: A Python library for serializing (pickling) objects, including models.
 - **joblib**: More efficient for large models, especially those with NumPy arrays. It is faster than **pickle** for larger datasets.
- **File Formats:**
 - **.sav, .pkl**: Common file extensions for serialized models using **pickle** or **joblib**.
 - **.h5**: Often used for neural networks, especially with libraries like TensorFlow or Keras.

BONUS: Containerization for ML Models

- **Why Save the Environment?:** The environment (Python version, libraries, dependencies) in which a model is trained can significantly impact its performance. By containerizing the model and environment, we ensure that it runs exactly the same in production as it did during development.

To save the environment into a container, we can use a tool called [Docker](#).

- **Docker**: A tool that packages an application and its dependencies into a container, ensuring consistent performance across different environments.

<https://medium.com/@rparundekar/deploying-ml-models-using-containers-in-three-ways-14745af94043>

Chapter 5: ML Pipelines

Pipelines in **scikit-learn** streamline the workflow by bundling multiple preprocessing steps and model training into a single, cohesive process. They help automate and standardize machine learning workflows.

Why Are Pipelines Useful?

- **Consistency**: Pipelines ensure that the same sequence of transformations (e.g., imputation, encoding, scaling) is applied consistently to both training and test data, avoiding manual errors.

- **Efficiency:** Automates preprocessing and model training in one step, reducing code complexity and the chance of missing crucial steps.
- **Modularity:** Makes it easy to replace or update components (e.g., swapping an imputer) without rewriting the entire process.

Why Bundle Imputers, Encoders, and Scalers?

- **Reproducibility:** By bundling these steps, the exact same transformations are applied during both training and inference, preventing discrepancies.
- **Avoid Data Leakage:** Ensures that transformations like scaling and imputing are fit on the training data only, not the test set, preventing information from the test set from leaking into the model.

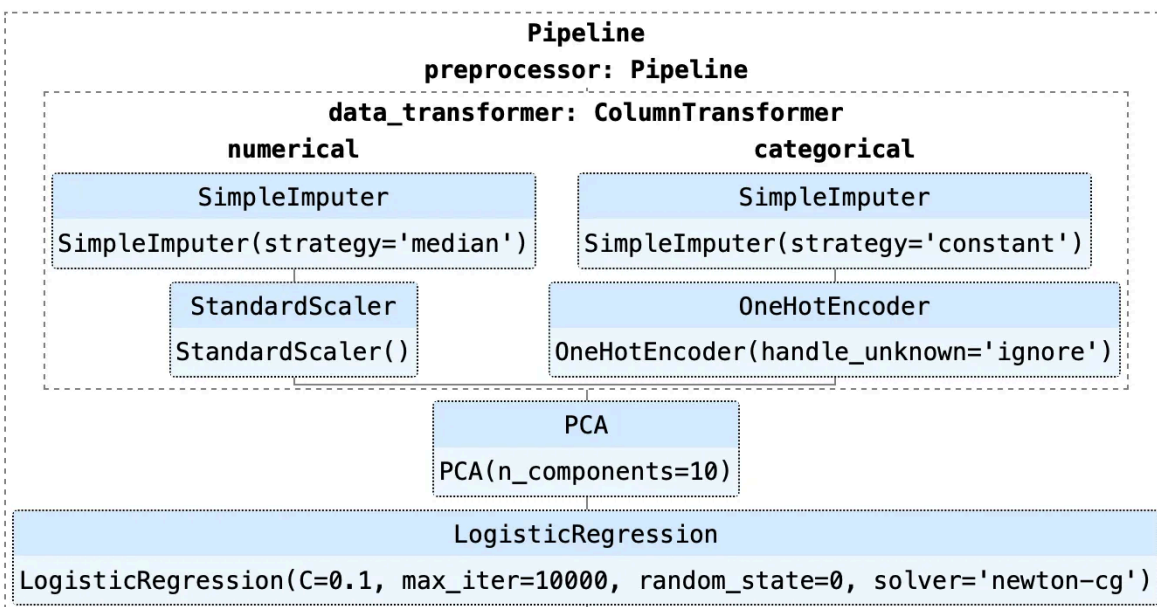
Common Pitfalls:

- https://scikit-learn.org/stable/common_pitfalls.html

Why Ensure Proper Test Set Transformation?

- **Correct Inference:** The test set must be transformed using the same pipeline (e.g., imputed values, scaled data) to ensure that the model receives data in the same format it was trained on, leading to accurate predictions during inference.

<https://towardsdatascience.com/are-you-using-pipeline-in-scikit-learn-ac4cd85cb27f>



Model Saving & Model Pipelines are all useful concepts within the larger MLOps paradigm

Chapter 6: K-Fold Cross-Validation

[K-Fold cross-validation](#) is a technique used to evaluate the performance of a model by splitting the dataset into multiple subsets, or "folds." The model is trained on $k-1$ folds and tested on the remaining fold, repeating this process k times. The results are averaged to give a more robust estimate of model performance. *It eliminates the use of a random_state and a train/test split.*

Why Is It Useful?

- **Reduces Overfitting:** By training and testing the model on different subsets of data, it provides a better indication of how the model will perform on unseen data holistically.
- **Utilizes the Entire Dataset:** Every data point is used both for training and validation, making full use of the dataset for better performance evaluation.
- **Limitations:** Since this approach trains an algorithm k times (usually 5) and then averages the metrics of each fit, it only works best with numeric, single, metrics like MSE/MAE/R2/Accuracy/AUC. You can't really make any of the visualizations.

I recommend doing Cross-Validation after your initial experiments on a few algorithms to confirm a winner on one or two metrics (MSE/R2 for regression, AUC/Accuracy for classification). Note this may not work well for imbalanced classification, since AUC/Accuracy are both less reliable.

Stratified K-Fold

- **Stratified K-Fold** ensures that each fold has a similar distribution of target classes, which is especially important for imbalanced datasets for classification.
- **When to Use It:** In classification tasks where the target variable has imbalanced classes (e.g., 90% of one class, 10% of another), Stratified K-Fold ensures each fold represents the class distribution accurately, leading to fairer model evaluation.

In a typical ML experiment, you'd use K-Fold cross-validation to test model generalization for regression and Stratified K-Fold when working with classification problems.

Chapter 7: Model Explainability

[Model explainability](#) helps understand how individual features contribute to a model's predictions, making the model's decision process more transparent and trustworthy.

Feature Importances for Trees

- **What:** In tree-based models (e.g., Random Forests, Gradient Boosting), feature importance indicates how much each feature contributes to the prediction by measuring how often it is used for splitting.
- **Why It's Useful:** Identifies key features driving the model, which helps in refining the model, feature selection, and providing insights into data patterns.

Coefficients for Linear Models

- **What:** In linear models (e.g., Linear Regression, Logistic Regression), the coefficients represent the strength and direction (positive or negative) of the relationship between a feature and the target variable.
- **Why It's Useful:** Coefficients allow for straightforward interpretation of feature impact, making it clear how each feature influences the prediction.
- **Limitations:** Using coefficients for feature impact are only applicable IF the numeric feature set was scaled. Otherwise, the coefficients are influenced by the range of the column.
- You can divide each coefficient by the sum of all the coefficients to create a decimal on the same scale as the feature importances (similar to what we did with LGBM's Feature Importances). Again, ONLY if the numeric feature set was scaled.

BONUS: SHAP Values

- **What:** SHAP (SHapley Additive exPlanations) values break down the prediction of a model into contributions from each feature, based on cooperative game theory. They quantify the contribution of each feature to the difference between the prediction and the average model output.
- **Why It's Useful:** [SHAP values](#) provide a more granular, consistent explanation of feature importance across different types of models, offering a clear, consistent measure of feature impact on individual predictions.

BONUS: Partial Dependence Plots (PDPs)

- **What:** [Partial Dependence Plots](#) show the relationship between a feature and the target variable by averaging the effect of that feature across the dataset, while holding other features constant.
- **Why It's Useful:** PDPs help visualize how changes in a feature affect model predictions, offering a broader understanding of the feature's impact on the outcome.

Chapter 8: Vocabulary

- Imputation
- [SimpleImputer](#)
- Mean/Median imputation
- Most-Frequent (Mode) imputation
- Encoding
- Label (Ordinal) Encoding
- One-Hot Encoding
- Dummy Variables
- Binning/Combining Categories
- Scaling
- ML Pipeline
- Pre-processing Pipeline
- K-Fold Cross Validation
- Stratification (Stratified K-Fold)
- Imbalanced Classification
- Pickling/Model Saving
- Containerization
- Joblib
- Docker
- Model Serving
- MLOps
- .sav, .pkl, .h5 file extensions
- Model Explainability
- “Explainable AI”
- Feature/Permutation Importance
- Bonus: SHAP Values
- Bonus: Partial Dependence Plots
- Bonus: Over/Under Sampling
- Bonus: SMOTE
- Bonus: Target Encoding